Alma Mater Studiorum – Università di Bologna

Bachelor's Degree in Management and Economics

Backtesting and Synthetic Market Evaluation of Trading Strategies

Supervisor:
Prof. Giovanni Cardillo

Candidate:
Jacopo Michelacci

Academic Year 2025/2026

Abstract

This thesis studies the problem of robust strategy optimization in systematic trading. The central question is how trading strategies can be optimized on past market data while limiting the risk of overfitting, but more specifically, which optimization procedure produces more reliable out-of-sample performance.

To address this question, the project develops an efficient research framework composed of a C++ backtesting and optimization engine linked to Python reporting and analysis tools. The C++ component is responsible for data processing, strategy simulation, trade reconstruction, and grid-based parameter optimization. The Python component is used for plotting, report generation, and performance analysis. This separation allows the computationally intensive parts of the workflow to remain efficient and independent, while keeping the analytical layer flexible.

The empirical analysis uses the same discretionary optimization procedure under two different data-generation settings. The benchmark approach optimizes the strategy on the historical in-sample period, defined as the first 70% of the original series. The alternative approach optimizes the strategy on synthetic markets generated from the same in-sample window: multiple synthetic paths are produced, the strategy is evaluated across them, and parameter selection is based on average performance rather than on a single historical path. In both cases, parameter selection is performed using discretion, following basic robustness principles rather than blindly selecting the single best result. The selected parameters are then frozen and tested on the same out-of-sample period, corresponding to the final 30% of the original historical series. The comparison focuses on the gap between optimization performance and out-of-sample performance: a smaller deterioration is interpreted as evidence of lower overfitting.

[will put the results here]

Contents

Chapter 1

Introduction

1.1 Motivation

Systematic trading strategies are rules that transform market data into trading decisions. Their appeal is straightforward: if a set of rules can identify repeatable market patterns, or possible market inefficiencies, it may be used to improve returns, control risk, and reduce dependence on discretionary judgement. Most strategies contain parameters, such as lookback windows, thresholds, or position-sizing rules. The process of choosing these parameters is known as strategy optimization, and it is a central part of algorithmic trading research. When performed carefully, optimization can help identify more effective configurations; when performed naively, it can lead to misleading results and poor live performance.

The difficulty is that better historical performance does not automatically imply better future performance. In financial markets, the familiar warning that past returns are not indicative of future returns is not a legal formality only; it reflects a real statistical problem. A strategy can perform well in a backtest because it captures a genuine and persistent market effect, but it can also perform well simply because its parameters were fitted too closely to the specific historical sample being tested. This second case is the problem of overfitting, which is perhaps the most common and insidious challenge that one can tackle in strategy optimization.

1.2 The overfitting problem

A naive approach to strategy development would be to optimize a strategy on the full available historical dataset and then deploy the best-performing configuration. This is problematic because the same data is used both to choose the parameters and to judge the quality of the strategy. If many parameter combinations are tested, some of them are likely to look good by chance, even if the underlying rule has little predictive value. The more parameter combinations and strategy variations

are tested, the greater the probability of finding a configuration that performed well on the historical sample by chance rather than because of a robust underlying effect. Such a configuration may produce an impressive backtest, but fail when applied to new market data.

Overfitting can therefore be understood as a loss of generalization. The strategy becomes too adapted to the noise, anomalies, and specific sequence of events observed in the historical sample. As a consequence, the more a strategy is overfitted, the less likely it is that its backtested performance will be repeated in the future.

A common response to this issue is to split the historical data into two parts. The first part, usually called the in-sample period (IS from now on), is used to develop and optimize the strategy. The second part, called the out-of-sample period (OOS from now on), is kept separate and used only after the parameters have been selected. In this thesis, the benchmark approach uses a 70% in-sample and 30% out-of-sample split, where the out-of-sample period corresponds to the most recent part of the original historical series. This reflects the practical objective of testing whether a strategy optimized on the past would have remained effective on later unseen data.

1.3 Research question

The central question of this thesis is whether a synthetic-market-based optimization procedure can improve robustness compared with the standard historical in-sample optimization approach. Both methods use the same general optimization logic, but they differ in the data on which the optimization is performed.

In the benchmark case, the strategy is optimized directly on the historical in-sample period. In the synthetic case, the in-sample period is first used to generate multiple synthetic market paths. The strategy is then optimized across these generated paths, and parameter selection is based on average results rather than on one realized historical path. The selected parameters from both approaches are finally tested on the same real out-of-sample period. This makes the comparison focused: the objective is not to prove that a strategy is profitable, but to evaluate which optimization procedure loses less performance when moving from optimization to OOS testing.

1.4 Approach and contribution

The research question requires both a clear testing procedure and code that can run many strategy tests efficiently. For this reason, I developed a custom research framework composed of a C++ backtesting and optimization engine connected to Python reporting and analysis tools. The C++ component handles the computationally intensive parts of the workflow, including data processing, strategy simulation, trade reconstruction, and grid-based optimization. Python is used for report

generation and plotting.

The framework makes it possible to compare the two optimization approaches directly. In both cases, the same strategy, data split, performance metrics, and out-of-sample test period are used. The only difference is where the optimization results come from: in the benchmark case, they come from the historical in-sample data; in the alternative case, they come from multiple synthetic paths generated from that same in-sample data. The main comparison is therefore the IS-OOS performance gap produced by each method.

Chapter 2

Methodology

2.1 Overview

The methodology is designed to compare two ways of optimizing trading strategies while keeping the final test as clean as possible. The first method is the standard and also our benchmark: optimize the strategy on past historical data and then test it on later unseen data. The second method uses synthetic markets: the synthetic paths are generated from the same IS period, the strategy is optimized across those paths, and the selected parameters are then tested on the same real OOS period.

The important point is that both methods are evaluated on the same final historical window. This makes the comparison focused on the optimization process rather than on differences in the test data. The synthetic approach is considered useful only if the parameters selected from synthetic paths transfer better to the real OOS period.

2.2 Data split

The historical dataset is split chronologically. The first part of the series is used as the IS period, while the final part is held out as the OOS period. In the experiments, the standard split is 70% IS and 30% OOS.

The IS data is used for strategy optimization and, in the synthetic approach, for generating synthetic market paths. The OOS data is not used during parameter selection. It is only used after the parameters have been chosen, in order to evaluate how the strategy behaves on unseen historical data.

This chronological split is important because financial data is time-ordered. Randomly shuffling observations would not represent the real situation faced by a trader, where parameters are chosen using past information and then applied to future market data.

All comparisons are performed on the same asset and the same OOS period.

2.3 Backtesting model

The backtester operates on OHLCV bars. Each bar contains open, high, low, close, volume, and timestamp information. Strategy logic is evaluated after a bar is available, and normal market orders generated by the strategy are filled at the open of the following bar.

This execution rule is used to avoid lookahead bias. A strategy is not allowed to observe a bar and then trade at a price from that same bar. For example, if a signal is generated after observing bar t, the earliest normal execution price is the open of bar $t + 1$.

The exception is stop-loss execution. Stop losses are treated as protective orders that are already active once a position has been opened. Therefore, if the high or low of a bar crosses the stop level, the stop is considered triggered inside that bar and is filled at the stop price. This is different from a new strategy signal, because the stop level was known before the bar was completed.

The equity curve is marked once per bar using the close price:

$$\text{Equity}_t = \text{Cash}_t + \text{Open Position}_t \cdot \text{Close}_t.$$

Therefore, drawdowns computed from the equity curve are close-to-close drawdowns. Intrabar high and low prices are not used to mark the equity curve, but they are used for stop-loss execution.

2.4 Stop-loss execution

Stop losses are treated as protective orders attached to open positions. This reflects the way a live system would typically work: after an entry is filled, a stop order can be placed at the broker and remain active while the position is open.

Because the available data is OHLCV bar data, the exact intrabar path is unknown. The backtester therefore uses the following convention: a long stop loss is considered triggered if the bar low is less than or equal to the stop price, while a short stop loss is considered triggered if the bar high is greater than or equal to the stop price. When triggered, the stop is filled at the stop price.

For a percentage stop loss, the stop level is computed from the entry price. For a long position:

$$\text{Stop Price} = \text{Entry Price} \cdot (1 - \text{Stop Percentage}).$$

For a short position:

$$\text{Stop Price} = \text{Entry Price} \cdot (1 + \text{Stop Percentage}).$$

The framework also supports a notional stop distance. In that case, the stop is placed at a fixed price distance from the entry. If both a percentage stop and a notional stop are provided, the backtester uses the tighter stop.

This stop-loss model is an approximation. It assumes that a stop is filled at the stop price once the bar high or low crosses it, and it does not model the exact intrabar sequence of prices. Transaction costs, including fees or an assumed slippage component, can be included by specifying a cost in basis points at the start of the backtest run.

2.5 Position and trade accounting

The backtester tracks both the net open position and the individual open lots. The net position represents the total current exposure. The open-lot log stores the individual entries that are still open.

This distinction is useful when stacking is allowed. If two buy orders are filled separately, they increase the same net long exposure, but they remain two distinct open lots. This makes it possible to track independent entry prices and independent stop losses.

When an order reduces an existing position, open lots are closed using FIFO. For example, if two long lots of quantity 1 are open and a sell order of quantity 1.2 is filled, the first lot is closed completely and the second lot is reduced to quantity 0.8.

This accounting also affects trade reconstruction. A trade is considered closed when an open lot, or part of an open lot, is closed by an opposite-side order or by a stop loss.

2.6 Performance metrics

The framework computes several metrics to evaluate each backtest:

- **Net profit**: final equity minus initial capital.
- **Average trade**: average profit or loss per completed trade.
- **Sharpe ratio**: annualized mean return divided by return volatility.
- **Maximum drawdown**: largest percentage decline from a previous equity peak.
- **Number of trades**: total number of completed trades.
- **Trades per year**: number of completed trades normalized by the length of the test period.

These metrics are used both during optimization and during final OOS evaluation. The purpose is not to rely on a single number only, but to evaluate a strategy on risk-adjusted return and frequency metrics.

2.7 Measuring overfitting

Overfitting is assessed by comparing optimization performance with OOS performance. For each method, the selected parameters have one performance value during optimization and one performance value on the real OOS period. If performance is much worse OOS, the optimized parameters likely captured too much of the specific optimization sample.

For a metric M where higher values are better, the performance gap is defined as:

$$\text{Performance Gap} = M_{opt} - M_{OOS}.$$

A smaller gap means that less performance was lost when moving from optimization to unseen data. In this thesis, the synthetic optimization method is considered less overfit if it produces a smaller IS-OOS gap than the historical benchmark on the same strategy and OOS period.

For metrics where lower values are better, such as maximum drawdown, the interpretation is reversed: the relevant deterioration is an increase from optimization drawdown to OOS drawdown.

2.8 Optimization protocol

Optimization is performed using grid search. A parameter grid is defined, the strategy is backtested for each parameter value, and the resulting metrics are recorded. The optimizer is implemented in C++ so that repeated backtests can be run efficiently.

Parameter selection is not intended to be a blind maximization of one metric. A parameter value that produces the best result IS may simply be fitted to that specific historical window. For this reason, the optimization plots are inspected with robustness in mind. The goal is to prefer parameter regions that show low variation across nearby values, rather than isolated peaks.

In the historical benchmark, the grid search is run directly on the historical IS period. In the synthetic approach, the same strategy and parameter grid are evaluated on multiple synthetic paths generated from the IS data. The optimization result is then based on mean performance across synthetic paths.

After parameter selection, the chosen parameters are frozen. Both the historically optimized version and the synthetically optimized version are then tested on the same real OOS period.

2.9 Implementation

The framework is implemented using C++ and Python. C++ is used for data loading, strategy execution, backtesting, optimization, and synthetic market generation. Python is used for plotting and report generation.

The C++ code was written with runtime efficiency in mind. The backtester keeps the main simulation loop in C++, avoids writing full report files during optimization runs, and reuses already-loaded data when evaluating multiple parameter values. The indicator interface uses the curiously recurring template pattern (CRTP), avoiding virtual dispatch in repeated indicator updates. Strategy objects are still accessed through a simple polymorphic interface, which keeps the backtesting interface flexible.

All experiments are run with the C++ code compiled in release mode, enabling compiler optimizations. The code is not parallelized, so each optimization run is executed sequentially. On the local test machine, a MacBook Air with an Apple M1 processor and 8 CPU cores, one optimization backtest run (daily data, 2000-2026) takes approximately 1.2–1.8 ms in the tested configurations.

Chapter 3

Trading Strategies

3.1 Overview

The empirical part of the thesis uses rule-based trading strategies. The goal is not to prove the profitability of a trading strategy, but to compare how different optimization procedures affect the same strategy when tested OOS. For this reason, the strategies are intentionally simple and interpretable.

Two strategy families are implemented in the framework: a moving-average crossover strategy and a standard-deviation mean-reversion strategy. They represent two common types of systematic trading logic. The first is trend-following, while the second is mean reverting.

3.2 Common strategy structure

All strategies follow the same execution interface. At each bar, the strategy receives the current market observation and the current portfolio context. It can then produce one or more order events. The backtester is responsible for executing those orders according to the execution rules described in the methodology chapter.

Strategies can be configured to allow or disable long trades, short trades, and stacking. Stacking means that a strategy is allowed to add to an existing position in the same direction. If stacking is disabled, the strategy cannot increase an already open position on the same side.

In the experiments, a fixed position size is used. This keeps the comparison focused on the optimization procedure and the trading rule, rather than on the effects of dynamic position sizing or compounding. Altough it could be argued that a fixed fractional positon sizing could highlight more realistically the effects of the 2 optimizations. As mentioned the framework also supports dynamic sizing based on current equity values, but this is not the sizing method used for the thesis optimization or results.

Both strategies can attach a hard stop loss to newly opened positions. The stop can be defined either

as a percentage move from the entry price or as a fixed notional price distance. Stop-loss execution is handled by the backtester at the open-lot level.

3.3 Moving-average crossover strategy

The moving-average crossover strategy is a trend-following rule. It compares a fast moving average with a slow moving average. The fast average reacts more quickly to recent price changes, while the slow average represents a smoother estimate of the price trend.

Let $MA_f(t)$ be the fast moving average at time t, and $MA_s(t)$ be the slow moving average. A long signal is generated when the fast moving average crosses above the slow moving average:

$$MA_f(t) \geq MA_s(t) \quad \text{and} \quad MA_f(t-1) < MA_s(t-1).$$

A short signal is generated when the fast moving average crosses below the slow moving average:

$$MA_f(t) \leq MA_s(t) \quad \text{and} \quad MA_f(t-1) > MA_s(t-1).$$

The main parameters of this strategy are the fast moving-average length, the slow moving-average length, and the stop-loss value.

3.4 Standard-deviation mean-reversion strategy

The standard-deviation mean-reversion strategy is a contrarian rule. Instead of following the direction of recent movement, it looks for unusually large one-bar moves and assumes that part of the move may revert.

At each bar, the strategy computes the last price move:

$$\Delta P_t = P_t - P_{t-1}.$$

It then computes a rolling standard deviation of recent price moves. The latest move is standardized by dividing it by this rolling standard deviation:

$$Z_t = \frac{\Delta P_t}{\sigma_t}.$$

where σ_t is the rolling standard deviation of x recent price moves.

A long signal is generated when the standardized move is below a lower threshold:

$$Z_t \leq \theta_L.$$

A short signal is generated when the standardized move is above an upper threshold:

$$Z_t \geq \theta_U.$$

The intuition is that a large negative move may be followed by a rebound, while a large positive move may be followed by a pullback. The main parameters of this strategy are the standard-deviation lookback length, the lower threshold, the upper threshold, and the stop-loss value.

3.5 Role in the thesis

The two strategies are used as test cases for the optimization methodology. They are deliberately simple enough to make their behaviour understandable, but different enough to test the optimization procedure on both trend-following and mean-reversion logic.

The exact parameter grids and selected parameter values are discussed in the optimization and results chapters.

Chapter 4

Synthetic Market Generation

4.1 Purpose of synthetic markets

The central problem of this thesis is not only whether a strategy performs well on one historical sample, but whether the chosen parameters are robust. A single historical price series gives only one realised path of the market. If the optimization is based only on that path, the selected parameters may fit details that are specific to that sample and may not repeat in the future.

Synthetic markets are used to create alternative price paths that preserve some basic statistical properties of the original data, while still producing different possible market evolutions.

In the thesis workflow, synthetic paths are generated from the historical IS data. The strategy is then optimized across those paths, using average performance across simulations instead of performance on one single historical IS sample. The final comparison is still made on the real OOS period, so the synthetic data is used only during the optimization phase.

4.2 General structure

Each synthetic path keeps the same timestamps and number of observations as the original data. The generator copies the structure of the original OHLCV series and then replaces the price values with simulated prices. Volume was not modeled nor considered in the synthetic generation as the strategies chosen operate without it.

All methods start from the first observed close price. For each following bar, the previous synthetic close is used as the next synthetic open. A synthetic close is then generated, and the high and low are constructed around the synthetic open and close. This keeps the generated series in a standard OHLCV format.

The generator currently implements three methods:

- a basic geometric Brownian motion baseline;
- a rolling-volatility geometric Brownian motion;
- a rolling-volatility geometric Brownian motion with threshold constraints.

In the synthetic path comparison shown in this chapter, each generator produces 50 paths. The rolling methods use a 30-bar window for volatility estimation. The threshold-constrained version uses the same 30-bar volatility window and a threshold multiplier of $k = 10$.

In the figures, the red line represents the original historical market path, while the gray lines represent the generated synthetic paths.

4.3 Geometric Brownian motion baseline

The first method is a basic geometric Brownian motion (GBM). It is included as a simple benchmark because it is easy to interpret and widely used as a first approximation for price simulation.

In continuous time, GBM is usually written as:

$$dP_t = \mu P_t dt + \sigma P_t dW_t,$$

where P_t is the price, μ is the drift, σ is the volatility, and dW_t is the Brownian motion shock. The exact discrete-time solution over one time step is:

$$P_t = P_{t-1} \exp\left((\mu - 0.5\sigma^2)\Delta t + \sigma\sqrt{\Delta t} z_t\right), \qquad z_t \sim N(0,1).$$

In the implementation, one time step corresponds to one bar of the input data, so $\Delta t = 1$.

The method first computes historical close-to-close log returns:

$$r_t = \log\left(\frac{P_t}{P_{t-1}}\right).$$

The mean and standard deviation of these returns are used as the per-bar drift and volatility estimates. The next synthetic close is therefore generated as:

$$P_t^{syn} = P_{t-1}^{syn} \exp\left((\mu - 0.5\sigma^2) + \sigma z_t\right),$$

where z_t is drawn from a standard normal distribution. The time step is one bar, so there is no explicit dt term in the implementation. The estimates are already expressed in the frequency of the input data.

This baseline is intentionally simple. Its main limitation is that it uses one global volatility estimate for the whole sample. Real markets usually show periods of higher and lower volatility, so a constant volatility model can produce paths that are too smooth in turbulent periods and too volatile in calm periods.

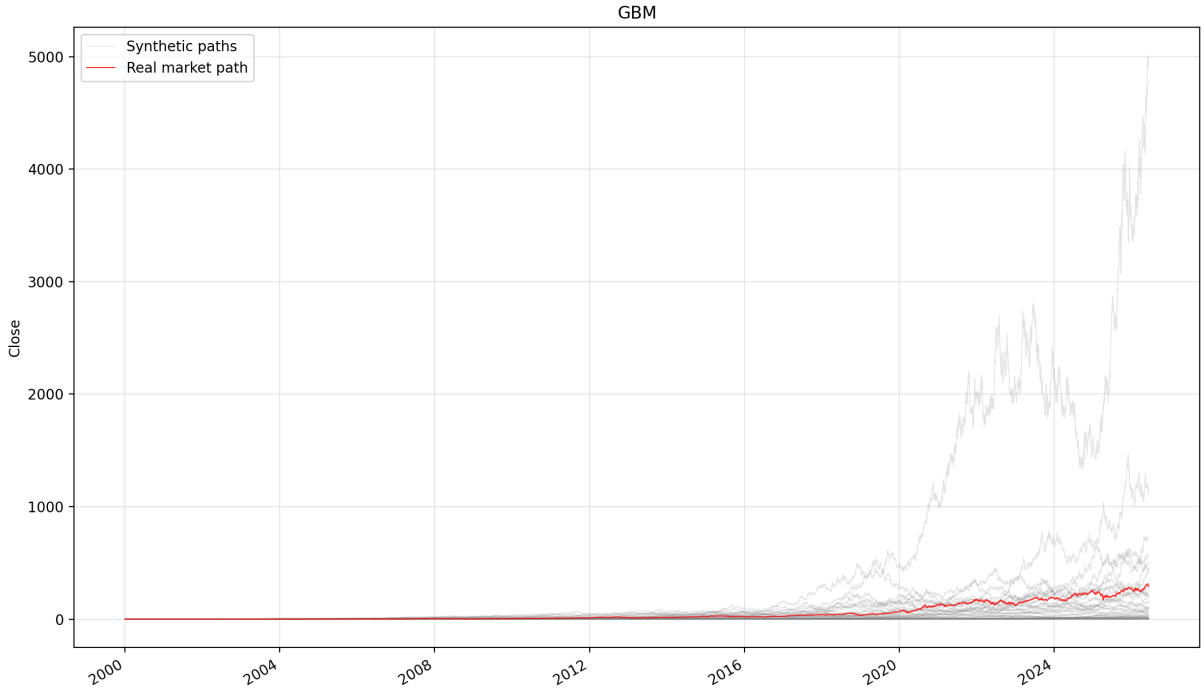

Figure 4.1: Synthetic paths generated with the basic GBM method.

The basic GBM paths move far away from the original series. This makes them less useful for optimization, because they no longer resemble the IS market environment closely enough.

4.4 Rolling-volatility GBM

The second method keeps the same GBM structure but replaces the global volatility estimate with a rolling one. Instead of using one standard deviation for the entire sample, the volatility used at each bar is estimated from the most recent returns in a rolling window. In the experiments shown here, this window is 30 bars.

The synthetic close is still generated from:

$$P_t^{syn} = P_{t-1}^{syn} \exp\left((\mu - 0.5\sigma_t^2) + \sigma_t z_t\right),$$

but σ_t now changes over time. This makes the synthetic paths closer to the structure of the original market, because volatility clusters are partly preserved. If the original IS data contains a volatile region, the generated paths around that region also tend to have larger moves.

The drift μ is kept as a global per-bar estimate. This keeps the method simple and avoids adding too many degrees of freedom. The main feature of this generator is the changing volatility, not a changing expected return.

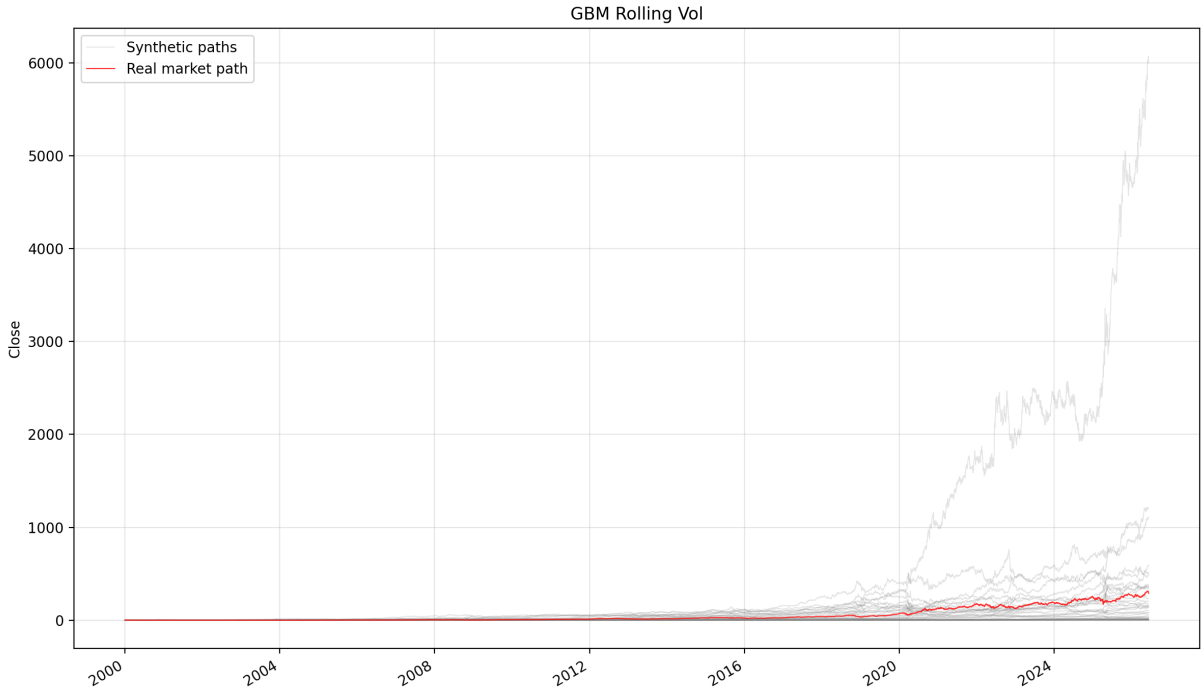

Figure 4.2: Synthetic paths generated with the rolling-volatility GBM method.

The rolling-volatility version is more realistic because volatility changes through time. It is not always more volatile than the basic GBM: it is calmer in low-volatility regions and more volatile in turbulent ones. However, the paths can still drift too far from the original series.

4.5 Threshold-constrained rolling-volatility GBM

The third method adds a constraint to the rolling-volatility generator. At each bar, the generator builds a local price band around the original historical price level. The width of the band is based on the rolling volatility estimate:

$$P_{t-1}^{orig} \exp(-k\sigma_t) \leq P_t^{syn} \leq P_{t-1}^{orig} \exp(k\sigma_t),$$

where k is the threshold multiplier.

If a sampled synthetic close falls outside this band, the value is rejected and sampled again. If the generator cannot find an accepted value after a fixed number of attempts, the value is clipped to the nearest boundary. This prevents extreme synthetic paths from drifting too far away from the original series while still allowing random variation around it.

This method is useful for robustness testing because it creates alternative paths that remain locally anchored to the original market regime. It is less free than the pure GBM generators, but it can produce synthetic data that is easier to compare with the historical IS sample.

Figure 4.3: Synthetic paths generated with the threshold-constrained rolling-volatility GBM method.

The threshold-constrained version is the most useful for this thesis. It keeps randomness, but remains linked to the original market by construction. The generated paths are therefore alternative versions of the same broad market environment, rather than unrelated price histories.

4.6 Synthetic high and low construction

The strategy backtester works with OHLCV bars, not only close prices. For this reason, the generator also creates synthetic highs and lows. The method estimates the historical size of the upper and lower bar extensions. For each bar, the upper extension is measured relative to the larger of the open and close, while the lower extension is measured relative to the smaller of the open and close. As below:

$$\text{High}_t \geq \max(\text{Open}_t, \text{Close}_t), \qquad \text{Low}_t \leq \min(\text{Open}_t, \text{Close}_t).$$

For the basic GBM, the generator uses global mean and standard deviation estimates for these extensions. For the rolling-volatility methods, the generator uses rolling estimates. This makes the intrabar range larger during high-volatility regions and smaller during low-volatility regions.

Chapter 5

Optimization

This chapter describes the parameter optimization process and the discretionary selection of robust parameter regions to reduce overfitting risk.

Chapter 6

Results

This chapter reports and discusses the empirical results obtained from historical backtests, optimization experiments, and synthetic market comparisons.

Chapter 7

Conclusion

This chapter summarizes the main findings, limitations, and possible future improvements.